POSTS AND TELECOMMUNICATIONS INSTITUTE OF TECHNOLOGY

Intelligent System Development

Assignment 02

From Data Representation to a Deployable Intelligent System

"A model is only as reliable as the data and evaluation process behind it."

Three required applications

1. Diabetes prediction
2. House-price prediction
3. E-commerce customer-interest discovery

Dinh Que Tran, Ph.D., Assoc. Prof.

Department of Information Technology
Posts and Telecommunications Institute of Technology (PTIT)

Email: quetd@ptit.edu.vn, tdque@yahoo.com

2026

1 Assignment Overview

Submit: Before 11:30PM, 07/08/2026
Prerequisite: Lecture 02 — Data Representation.

The purpose of Assignment 02 is to move from the representation of data to the development of complete intelligent applications. Students must develop **three applications** using real datasets obtained from Kaggle:

1. Diabetes prediction;
2. House-price prediction;
3. E-commerce customer behavior and interest discovery.

The three applications must demonstrate that different forms of real-world data require appropriate computational representations.

The overall pipeline is:

$$\boxed{\text{Raw Data} \rightarrow \text{Understand} \rightarrow \text{Clean} \rightarrow \text{Represent} \rightarrow \text{Learn} \rightarrow \text{Evaluate} \rightarrow \text{Persist} \rightarrow \text{Deploy}}$$

The assignment therefore does **not** mean simply training three models.

Students must demonstrate how data becomes a numerical representation that can be processed by a machine-learning model.

2 Connection with Lecture 02 — Data Representation

Lecture 02 introduced the principle:

$$\boxed{\text{Real-world data} \rightarrow \text{numerical representation} \rightarrow \text{computational model}}$$

Students must explicitly identify the representation used in each application.

Application	**Raw data**	**Computational representation**
Diabetes	CSV / tabular	Feature vector and feature matrix
House price	CSV / tabular	Feature vector and feature matrix
E-commerce behavior	CSV + customer comments/reviews	Tabular feature vectors + text token/embedding representations

For tabular data:

$$x = [x_1, x_2, \ldots, x_d]^T \in \mathbb{R}^d$$

and a dataset becomes a matrix:

$$X \in \mathbb{R}^{N \times d}.$$

For text, students should recognize the transformation:

$$\text{Text} \rightarrow \text{Tokens} \rightarrow \text{Token IDs} \rightarrow \text{Embeddings}.$$

For a batch of text embeddings:

$$E \in \mathbb{R}^{B \times T \times d}.$$

Important: Students must explain the representation of their data before training the model. Showing only the model architecture is not sufficient.

3 Three Required Applications

Each student/team must develop all three applications.

3.1 Application 1 — Diabetes Prediction

Select an appropriate diabetes-related Kaggle dataset.

The application should predict whether a patient belongs to a diabetes-related target class. Typical input features may include:

- age;
- BMI;
- blood pressure;
- glucose-related measurements;
- insulin;
- pregnancy-related measurements;
- other available clinical features.

The students must define:

$$X = \text{patient features}$$

and

$$y = \text{diabetes class}.$$

This is primarily a **classification problem**.

Representation requirement
Show at least one original CSV record, the corresponding feature vector, the complete feature matrix shape, and the final tensor or numerical representation supplied to the model.

3.2 Application 2 — House-Price Prediction

Select an appropriate house-price Kaggle dataset.

The application should predict the price of a house from its characteristics. Possible features include:

- living area;
- number of bedrooms;
- number of bathrooms;
- location;

- number of floors;
- year built;
- parking or garage information;
- other available property attributes.

The students must define:

$$X = \text{house features}$$

and

$$y = \text{house price.}$$

This is primarily a **regression problem**.

Representation requirement
Students must identify numerical and categorical variables and show how they become a single computational feature vector.
For example:

$$x = [x_{\text{area}}, x_{\text{bedroom}}, x_{\text{bathroom}}, x_{\text{location}}, \ldots]^T.$$

3.3 Application 3 — E-commerce Customer Behavior

Select an appropriate Kaggle e-commerce/customer-behavior dataset.

The objective is to analyze customer behavior and use available customer information and, where possible, **customer comments, reviews, feedback, or text descriptions** to discover interests.

The application may address questions such as:

- What product category is the customer interested in?
- What type of product is the customer likely to purchase?
- Can customer comments indicate product interests?
- Can customers be classified according to their interests?
- Can text feedback be used to predict customer preference?

A possible formulation is:

$$X = \text{customer behavioral features} + \text{text representation}$$

and

$$y = \text{customer interest / preference / category.}$$

The precise target must be determined from the selected dataset.

Text representation requirement
If customer comments or reviews are available, students must demonstrate the transformation:

$$\boxed{\text{Comment} \rightarrow \text{Tokens} \rightarrow \text{Token IDs} \rightarrow \text{Vector / Embedding}}$$

For example:

"I like wireless headphones"

may become

$$[t_1, t_2, t_3, t_4]$$

then

$$[x_1, x_2, x_3, x_4]$$

and eventually an embedding representation such as

$$E \in \mathbb{R}^{T \times d}.$$

Part I — Dataset and Problem Definition

For each of the three applications, students must provide:

1. Kaggle dataset name;
2. Kaggle URL;
3. number of observations;
4. number of attributes;
5. feature names;
6. data types;
7. target variable;
8. description of one observation;
9. application objective.

Students should include a compact table:

Application	Rows	Features	Target
Diabetes			
House price			
E-commerce			

Part II — Data Inspection and Understanding

For each dataset, students must perform:

```
df.shape
df.head()
df.info()
df.describe()
df.isna().sum()
df.duplicated().sum()
```

Students must identify:

- numerical columns;
- categorical columns;
- text columns;
- target column;
- missing values;
- duplicated records;
- invalid values;
- outliers;
- class imbalance where applicable.

Students must explain the results.

Part III — Data Cleaning

For each application, students must document the cleaning procedure.

The procedure may include:

1. removing or correcting duplicates;
2. handling missing values;
3. handling invalid values;
4. detecting outliers;
5. correcting data types;
6. removing irrelevant columns;
7. handling inconsistent categorical values;
8. cleaning text comments.

For text data, cleaning may include:

- removing unnecessary whitespace;
- handling missing comments;
- normalizing text;
- handling punctuation where appropriate;
- tokenization.

Students must explain WHY each operation is performed.

Part IV — Data Representation

This is the **central connection** to Lecture 02.

For each application, **students must explicitly show:**

Raw representation → Clean representation → Numerical representation → Model input

3.4 Tabular representation

For diabetes and house-price data, **students should show:**

$$x_i = [x_{i1}, x_{i2}, \ldots, x_{id}]$$

and

$$X = \begin{bmatrix} x_1^T \\ x_2^T \\ \vdots \\ x_N^T \end{bmatrix} \in \mathbb{R}^{N \times d}.$$

Students must report:

- original dataframe shape;
- feature matrix shape;
- data type;
- feature encoding;
- normalization/scaling;
- final model-input shape.

3.5 Categorical representation

For categorical variables, **explain how values such as**

$$\{\text{Urban}, \text{Suburban}, \text{Rural}\}$$

become numerical representations.

For example, one-hot encoding:

$$\text{Urban} \rightarrow [1, 0, 0].$$

3.6 Text representation

For customer comments:

$$\text{raw comment} \rightarrow \text{tokens} \rightarrow \text{IDs} \rightarrow \text{embeddings}.$$

Students must report:

$$B, \quad T, \quad d$$

and the final representation:

$$E \in \mathbb{R}^{B \times T \times d}.$$

Representation is part of the ML solution.
Students receive marks not only for obtaining good accuracy, but also for correctly explaining how raw data becomes a computational representation.

Part V — Exploratory Data Analysis

For every application, students must include meaningful visualizations. At minimum:

1. target distribution;
2. important feature distributions;
3. relationships between features and target;
4. correlation analysis where appropriate.

For e-commerce data, additionally analyze:

- customer purchase behavior;
- product categories;
- frequency of interests;
- comment/review characteristics;
- frequent terms or keywords where appropriate.

For every major visualization students must provide:

Observation: What does the figure show?

Interpretation: What does it mean?

ML implication: Why does it matter?

Part VI — Train, Validation, and Test Data

Students should use:

$$D = D_{\text{train}} \cup D_{\text{validation}} \cup D_{\text{test}}$$

with the sets kept independent.
A recommended split is:

$$70\%/15\%/15\%.$$

Alternatively:

$$80\%/20\%$$

may be used when justified.
For classification, stratification should be considered.
Students must explain why test data must not influence model training or preprocessing fitting.

Part VII — Model Development

Students must develop suitable ML models for each application.

3.7 Diabetes

This is a classification task.
Recommended models:

- Logistic Regression;
- Decision Tree;
- Random Forest;
- Support Vector Machine;
- KNN.

Five models must be compared.

3.8 House Price

This is a regression task.
Recommended models:

- Linear Regression;
- Ridge/Lasso Regression;
- Decision Tree Regressor;
- Random Forest Regressor;
- Gradient Boosting Regressor.

Five models must be compared.

3.9 E-commerce Customer Behavior

Students may formulate the task as classification or prediction depending on the selected dataset.
Six models must be compared.

- Logistic Regression;
- Decision Tree;
- Random Forest;
- SVM;
- text-based linear classifier;
- another justified model.

Students should compare a **tabular-only representation** with a representation that incorporates customer comments when the dataset supports this.

Part VIII — Model Evaluation

Classification

For diabetes and classification-based e-commerce prediction, report:

- Accuracy;
- Precision;
- Recall;
- F1-score;
- confusion matrix;
- ROC-AUC where appropriate.

The confusion matrix should be interpreted rather than simply displayed.

Regression

For house-price prediction, report:

- MAE;
- MSE;
- RMSE;
- R^2.

For example:

$$MAE = \frac{1}{N}\sum_{i=1}^{N}|y_i - \hat{y}_i|.$$

$$RMSE = \sqrt{\frac{1}{N}\sum_{i=1}^{N}(y_i - \hat{y}_i)^2}.$$

Students must explain what each metric means in the application.

Part IX — Model Comparison

Students must create a comparison table for each application.

For classification:

Model	Accuracy	Precision	Recall	F1
Model 1				
Model 2				
Model 3				
Model 4				

For regression:

Model	MAE	MSE	RMSE	R^2
Model 1				
Model 2				
Model 3				
Model 4				

Students must justify the selected final model.

Part X — Model Persistence

The final model must be saved so that it can be reused without retraining. For example:

```
import joblib

joblib.dump(
    pipeline,
    "model_pipeline.joblib"
)
```

Students must save:

- preprocessing;
- feature transformation;
- trained model;
- model configuration where necessary.

The deployed service must use exactly the same preprocessing logic as the model used during training.

Part XI — Web Deployment

Each application must be deployed as a simple web service. Students may use:

- FastAPI;
- Flask;
- or another justified REST framework.

The service should provide an endpoint such as:

```
POST /predict
```

The client sends the required features and receives:

```
{
    "prediction": "...",
    "confidence": 0.91
}
```

For house-price prediction:

```
{
    "predicted_price": 285000
}
```

For e-commerce interest discovery:

```
{
    "interest": "electronics",
    "confidence": 0.87
}
```

Students must demonstrate the API using a browser, Swagger UI, Postman, or another client.

Part XII — Mobile Deployment

Each application must also be demonstrated through a simple mobile interface.

The mobile application does not necessarily need to run the ML model directly on the device. A recommended architecture is:

Mobile UI → REST API → Preprocessing → ML Model → Prediction → Mobile UI

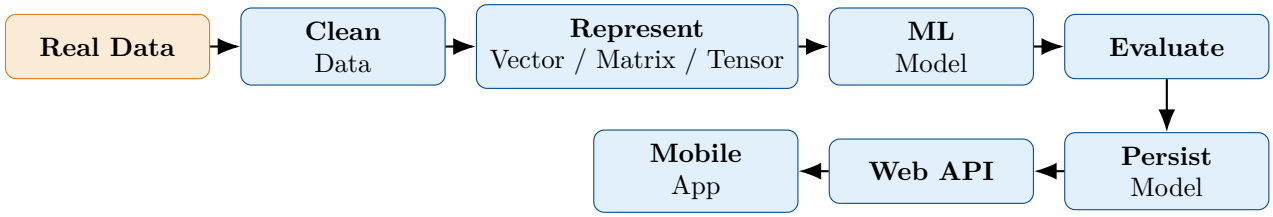
The mobile interface should allow the user to:

1. enter or select input information;
2. submit a prediction request;
3. display the prediction;
4. display an appropriate explanation or confidence value.

Students may implement the mobile client using an appropriate framework such as Android/Kotlin, Flutter, React Native, or another approved technology.

Complete Intelligent-System Architecture

The final system should demonstrate the complete path:



The architecture must be demonstrated for all three applications.

Required Deliverables

Each student must submit:

1. Three Jupyter notebooks:
   - diabetes;
   - house price;
   - e-commerce customer behavior.

2. Three trained and persisted ML pipelines.
3. Three web prediction services.
4. Three mobile demonstrations.
5. A final report.
6. Source code.
7. Dataset references.
8. README explaining how to reproduce the systems.

Report Structure

The report should contain approximately 10 pages.

1. Introduction and objectives
2. Data sources and application definitions
3. Data understanding and data quality
4. Data representation
5. Preprocessing and exploratory analysis
6. Model development
7. Evaluation and model comparison
8. Web and mobile deployment
9. Discussion
10. Conclusion

The three applications should be discussed using the same development framework so that their differences can be compared.

Mandatory Data-Representation Summary

The final report must include the following table:

Application	**Raw form**	**Numerical representation**	**Model input**
Diabetes	CSV / table	Feature vector / matrix	$B \times d$
House price	CSV / table	Encoded feature matrix	$B \times d$
E-commerce	CSV + comments	Tabular features + text vectors/embeddings	$B \times d$ and/or $B \times T \times d$

Students must explain every dimension in the reported shapes.

Discussion Questions

For each application, answer:

1. What does one observation represent?
2. What is the raw data representation?
3. What is the final numerical representation?
4. What do the dimensions of the feature matrix mean?
5. Which features required encoding?
6. Which features required normalization?
7. What information was lost during representation?
8. What information was preserved?
9. What preprocessing could cause data leakage?
10. Which model performed best?
11. Why was that model selected?
12. Which evaluation metric is most important?
13. How is the model persisted?
14. How does the Web service use the persisted model?
15. How does the mobile application communicate with the prediction service?

For the e-commerce application, additionally answer:

1. What information is contained in customer comments?
2. How are comments transformed into numerical data?
3. What do token IDs represent?
4. What do embedding vectors represent?
5. Which customer interests can be discovered?
6. Does text improve prediction compared with tabular features alone?

Assessment Rubric

Component	Weight	Key evidence
Problem understanding	10%	Clear application and target
Data quality	10%	Missing, duplicate, invalid data
Data representation	15%	Vector/matrix/tensor explanation
EDA	10%	Meaningful visualization
Preprocessing	10%	Reproducible pipeline
ML models	15%	Multiple models and comparison
Evaluation	10%	Correct metrics and interpretation
Web deployment	7.5%	Working prediction API
Mobile deployment	7.5%	Working mobile interface
Discussion and reproducibility	5%	Technical explanation and README

Final Requirement

The assignment is considered complete only when students can demonstrate all three systems:

Diabetes Prediction
↓
House-Price Prediction
↓
E-commerce Interest Discovery

and for each system:

Data → Clean → Represent → Train → Evaluate → Persist → Web → Mobile

The central learning objective is:

Students should understand not only how to train an ML model, but how raw real-world data is transformed into a reliable computational representation and ultimately into a usable intelligent service.

4 Required Report Structure

Each student must develop **three intelligent applications** using three different Kaggle datasets:

1. **Diabetes Prediction**
2. **House Price Prediction**
3. **E-Commerce Customer Behavior / Interest Discovery**

The three applications must demonstrate how different types of real-world data are transformed into computational representations and then used by machine-learning models.

The same development framework must be followed for all three applications:

Data → Understand → Clean → Represent → Learn → Evaluate → Persist → Deploy

The final submission consists of:

- one technical report;
- one Jupyter Notebook for each application;
- trained model files;
- preprocessing pipeline;
- deployment source code;
- screenshots demonstrating the web service;
- screenshots demonstrating the mobile application;
- source code repository.

5 Report Template

The report should contain approximately **10 pages**, excluding source code and appendices.

Students should use the following structure.

Cover Page

INTELLIGENT SYSTEM DEVELOPMENT

ASSIGNMENT 02

From Data Representation to Deployable Intelligent Systems

Student name:	.....
Student ID:	.....
Class:	.....
Team:	.....
Lecturer:	Dinh Que Tran, Ph.D., Assoc. Prof.
Semester:	I.2026

6 Executive Summary

Provide a short summary of the three intelligent applications.

Application	**Prediction / Task**	**Main Representation**
Diabetes	Diabetes classification	Feature vector / matrix
House Price	Price regression	Feature vector / matrix
Customer Behavior	Behavior / interest prediction	Feature vector / matrix

For each application, briefly state:

- dataset;
- problem;
- representation;
- selected model;
- main evaluation result;
- deployment method.

7 Connection to Lecture 02: Data Representation

Explain how real-world data becomes numerical data.

Students must explicitly identify:

$$\boxed{\text{Real-world object} \rightarrow \text{Raw data} \rightarrow \text{Numerical representation} \rightarrow \text{Tensor}}$$

For example:

Application	Raw Data	ML Representation
Diabetes	CSV	feature matrix
House Price	CSV	feature matrix
Customer Behavior	CSV / transactions	feature matrix / sequences

For every application, students must answer:

1. What does one row represent?
2. What does one column represent?
3. Which columns are input features?
4. Which column is the target?
5. Which features are numerical?
6. Which features are categorical?
7. How are categorical values encoded?
8. What is the final feature dimension?
9. What is the shape of the model input?

8 Application 1 — Diabetes Prediction

8.1 Problem Description

Describe the healthcare problem.

Template:

> The objective of this application is to predict whether a patient is likely to have diabetes based on ..... The prediction target is ..... The prediction can potentially support .....

Students must clearly define:

$$X = \text{patient features}$$

$$y = \text{diabetes class.}$$

8.2 Kaggle Dataset

Provide:

- dataset name;
- Kaggle URL;
- number of observations;
- number of features;
- target variable;
- feature descriptions.

Feature	Type	Meaning
Age	Numerical	.....
BMI	Numerical	.....
Glucose	Numerical	.....
.....	.....	.....

8.3 Data Understanding

Report:

- dataset shape;
- data types;
- missing values;
- duplicated records;
- invalid values;
- class distribution.

Include screenshots or tables from the notebook.

8.4 Data Cleaning

Explain:

- missing-value treatment;
- duplicate removal;
- invalid-value treatment;
- outlier analysis;
- categorical-value processing.

Do not only state what was done.
Explain **why** it was done.

8.5 Data Representation

Show the transformation:

CSV → DataFrame → clean feature matrix → scaled/encoded matrix → model input.

Report:

$$X \in \mathbb{R}^{N \times d}.$$

Example:

$$X = \begin{bmatrix} x_{11} & x_{12} & \cdots & x_{1d} \\ x_{21} & x_{22} & \cdots & x_{2d} \\ \vdots & \vdots & \ddots & \vdots \\ x_{N1} & x_{N2} & \cdots & x_{Nd} \end{bmatrix}.$$

State the actual values of N and d for the selected dataset.

8.6 Exploratory Data Analysis

Include at least:

- target distribution;
- feature distributions;
- important feature relationships;
- correlation analysis;
- at least three meaningful plots.

For every important plot:

Observation: What do you see?

Interpretation: What does it mean?

ML implication: Why is it relevant?

8.7 Model Development

Train at least four models.
Recommended:

- Logistic Regression;
- KNN;
- Decision Tree;
- Random Forest;
- SVM.

8.8 Evaluation

Report:

- Accuracy;
- Precision;
- Recall;
- F1-score;
- ROC-AUC where appropriate;
- confusion matrix.

Explain which metric is most important for diabetes prediction.

8.9 Model Selection

Explain:

> Which model would you select for deployment and why?

Consider:

- predictive performance;
- interpretability;
- computational cost;
- robustness;
- deployment constraints.

8.10 Deployment

Deploy the selected model as:

$$\boxed{\text{Web Service} + \text{Mobile Application}}$$

The web application should provide:

```
Input patient information
        |
        v
Preprocessing
        |
        v
ML Model
        |
        v
Prediction
```

The mobile application should provide a simple user interface for entering the same features and displaying the prediction.

Include screenshots.

9 Application 2 — House Price Prediction

9.1 Problem Description

Describe the problem of predicting house prices.

Define:

$$X = \text{house characteristics}$$

and

$$y = \text{house price.}$$

This application is a **regression problem**.

Students must explain why it is different from diabetes classification.

9.2 Dataset

Provide:

- Kaggle dataset name and URL;
- number of observations;
- number of features;
- target variable;
- numerical features;
- categorical features.

9.3 Data Understanding and Cleaning

Analyze:

- missing values;
- duplicates;
- invalid values;
- outliers;
- skewed numerical variables;
- categorical variables.

9.4 Representation

Show:

$$\text{Raw CSV} \rightarrow \text{DataFrame} \rightarrow \text{Clean Data} \rightarrow \text{Encoded/Scaled Features} \rightarrow X.$$

Report:

$$X \in \mathbb{R}^{N \times d}, \qquad y \in \mathbb{R}^{N}.$$

9.5 Exploratory Data Analysis

At minimum include:

- house-price distribution;
- relationship between important features and price;
- correlation analysis;
- relevant scatter plots;
- analysis of possible outliers.

9.6 Regression Models

Train at least four models, for example:

- Linear Regression;
- Ridge Regression;
- Decision Tree Regressor;
- Random Forest Regressor;
- Gradient Boosting Regressor.

9.7 Evaluation

Report appropriate regression metrics:

$$MAE = \frac{1}{N}\sum_{i=1}^{N}|y_i - \hat{y}_i|$$

$$MSE = \frac{1}{N}\sum_{i=1}^{N}(y_i - \hat{y}_i)^2$$

$$RMSE = \sqrt{MSE}.$$

Also report R^2.

Model	MAE	RMSE	R^2	Training Time
Linear Regression				
Ridge				
Decision Tree				
Random Forest				

9.8 Deployment

Deploy the selected house-price model on:

- a web interface;
- a mobile interface.

The user enters house characteristics and receives:

Predicted House Price

10 Application 3 — E-Commerce Customer Behavior and Interest

10.1 Problem Description

The objective is to analyze customer behavior and discover customer interests from e-commerce data.

Students must define a specific supervised-learning task.

Possible tasks include:

- predict whether a customer will purchase;
- predict whether a customer will return;
- predict customer segment;
- predict product-category interest;
- predict high-value customer behavior.

Students must select **one clearly defined target**.
The problem should be written as:

$$X = \text{customer behavioral features}$$

$$y = \text{behavior / interest target.}$$

10.2 Dataset

Provide:

- Kaggle dataset name and URL;
- number of customers/transactions;
- feature descriptions;
- target definition.

10.3 Customer Representation

Students should explicitly explain how a customer's behavior is converted into numerical features.
For example:

$$x_i = [\text{age}, \text{frequency}, \text{monetary value}, \text{sessions}, \text{cart actions}, \text{category counts}, \ldots]^T.$$

If transaction-level data are used, students should explain how multiple transactions are aggregated into a customer-level representation.
For example:

$$\text{Transactions} \rightarrow \text{Customer Profile} \rightarrow \text{Feature Vector}.$$

10.4 Data Cleaning

Investigate:

- missing customer information;
- duplicate transactions;
- invalid transactions;
- impossible prices or quantities;
- inconsistent categories;
- extreme purchasing behavior.

10.5 Interest Discovery

Students must identify important behavioral patterns.

Examples:

- frequently purchased categories;
- purchase frequency;
- average order value;
- recency;
- browsing behavior;
- cart behavior;
- customer activity.

A simple customer representation may be:

$$x_i = [R_i, F_i, M_i, C_{i1}, C_{i2}, \ldots, C_{ik}].$$

where:

- R_i = recency;
- F_i = purchase frequency;
- M_i = monetary value;
- C_{ij} = activity related to category j.

10.6 Model Development

Train at least four suitable models.

For a classification task:

- Logistic Regression;
- Decision Tree;
- Random Forest;
- SVM or KNN.

10.7 Evaluation

Report appropriate metrics:

- Accuracy;
- Precision;
- Recall;
- F1;
- ROC-AUC where appropriate;
- confusion matrix.

10.8 Business Interpretation

Students must answer:

> What customer behavior or interest has been discovered, and how could an e-commerce company use the prediction?

Possible applications include:

- personalized recommendations;
- targeted promotions;
- customer retention;
- product recommendations;
- marketing campaigns.

10.9 Deployment

Deploy the selected customer-behavior model as:

Web Application + Mobile Application

The system should accept customer information and return a predicted behavior or interest.

11 Comparison of the Three Intelligent Systems

Students must provide one integrated comparison.

	Diabetes	**House Price**	**Customer Behavior**
Problem type	Classification	Regression	Classification / selected task
Raw data			
Observation	Patient	House	Customer / transaction
Input shape			
Target			
Representation	Feature vector	Feature vector	Behavior vector
Best model			
Main metric			
Web deployment	Yes	Yes	Yes
Mobile deployment	Yes	Yes	Yes

Students must discuss:

1. How are the three datasets different?
2. How are their representations different?
3. Which preprocessing operations are common?
4. Which preprocessing operations are application-specific?
5. Why is the target representation different?

6. Why are different evaluation metrics required?
7. Which system is easiest to deploy?
8. Which system is most computationally demanding?

12 Deployment Architecture

All three applications must demonstrate a complete inference pipeline.

The deployment system should contain:

1. trained model;
2. preprocessing pipeline;
3. API endpoint;
4. input validation;
5. prediction function;
6. result returned to the client;
7. web user interface;
8. mobile user interface.

Students must explain why the same preprocessing used during training must also be used during inference.

13 Web Application

Each of the three applications must be exposed through a working web application.

The web application is not required to retrain the model. It must load the saved model and preprocessing pipeline and perform inference on new user input.

Students may use:

- FastAPI;
- Flask;
- Streamlit;
- another approved web framework.

13.1 Required Web Workflow

The web application should implement the following process:

User Input → Validation → Preprocessing → Saved Model → Prediction → Result

The application should contain:

1. input form;
2. appropriate input controls;
3. input validation;
4. prediction button;
5. prediction result;
6. confidence or probability when appropriate;
7. brief interpretation of the result.

13.2 Application-Specific Input

The input interface must correspond to the data representation developed during the ML experiment.

Diabetes prediction: The user enters the clinical/demographic features required by the selected dataset. The system returns the predicted diabetes class or probability.

House-price prediction: The user enters property characteristics such as area, number of rooms, location-related variables, or other features used by the selected dataset. The system returns the estimated house price.

Customer behavior: The user enters or selects customer characteristics and/or behavioral features. The system returns the predicted customer behavior, segment, purchase intention, or other target defined by the selected dataset.

13.3 Web Application Evidence

For each application, include at least:

1. input screen;
2. prediction/result screen;
3. an example of valid input;
4. an example of the returned prediction.

Use the following template for each application:

Web Application Screenshot — [Application Name]

Insert screenshot of the working web application here.

Figure explanation: Briefly describe the input, prediction, and interpretation shown in the screenshot.

14 Mobile Application

Each of the three ML applications must also be accessible through a mobile application.

The mobile application acts as a client of the deployed machine-learning service.

Possible technologies include:

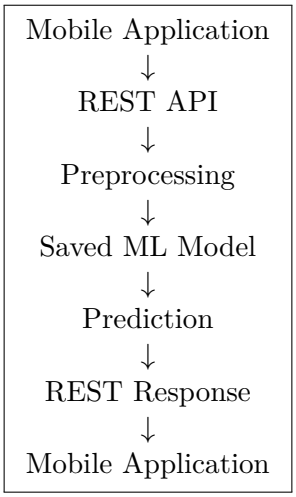
- Flutter;
- Android Studio;
- React Native;
- another approved mobile framework.

The mobile application does not need to contain the complete ML training process.

Instead, it should communicate with the deployed prediction service.

14.1 Mobile Architecture

The expected architecture is:



Students should clearly distinguish:

$$\text{Training} \neq \text{Inference}.$$

The model is trained during the experimental stage and saved. The mobile application performs inference using the deployed model.

14.2 Required Mobile Functionality

The mobile application should provide:

1. input screen;
2. input validation;
3. prediction request;
4. connection to the REST API;
5. prediction result;
6. confidence/probability when appropriate;
7. user-friendly result presentation.

14.3 Mobile Application Evidence

For each application, include:

1. mobile input screen;
2. prediction/result screen;
3. evidence that the application communicates with the prediction service.

Mobile Application Screenshot — [Application Name]

Insert screenshot of the mobile input and prediction screens here.

Figure explanation: Explain how the mobile application sends input to the API and displays the returned prediction.

15 Deployment and Inference

The deployment stage connects the machine-learning experiment with a usable software system.

For each application, students must demonstrate:

$$\boxed{\text{Saved Preprocessor} + \text{Saved Model} + \text{API} + \text{Web Client} + \text{Mobile Client}}$$

The deployed system should perform the following inference process:

$$\text{Raw User Input} \rightarrow \text{Validation} \rightarrow \text{Same Preprocessing} \rightarrow \text{Model} \rightarrow \text{Prediction}.$$

The preprocessing used during deployment must be the same preprocessing used during model training.

Important: Data Leakage

Students must not fit a new scaler, encoder, imputer, or other preprocessing component using test data or user input during deployment.

The deployed system must load the preprocessing pipeline learned from the training data.

16 Reproducibility

A major objective of Assignment 02 is to develop a reproducible machine-learning system. Another student should be able to reproduce the main experiment and run the deployed application.

For each of the three applications, provide:

- Python version;
- operating system;
- library versions;
- random seed;
- Kaggle dataset source;
- dataset version or download information;
- preprocessing procedure;

- feature representation;
- train/test split;
- model hyperparameters;
- evaluation metrics;
- saved preprocessing pipeline;
- saved model;
- API code;
- web application code;
- mobile application code.

Example:

```
import numpy as np
import random

RANDOM_SEED = 42

np.random.seed(RANDOM_SEED)
random.seed(RANDOM_SEED)
```

Students should also provide a `requirements.txt` or equivalent environment description.

17 Final Comparative Discussion

Students must compare the three applications rather than discussing them independently only.

The discussion should address the complete pipeline:

$$\boxed{\text{Data} \rightarrow \text{Representation} \rightarrow \text{Model} \rightarrow \text{Evaluation} \rightarrow \text{Deployment}}$$

Students must answer the following questions.

1. How are the three datasets different?
2. What does one observation represent in each dataset?
3. What is the target variable for each application?
4. What is the computational representation of each dataset?
5. Which application uses classification?
6. Which application uses regression?
7. How are categorical variables represented?
8. How are numerical variables represented and scaled?
9. Which data-quality problems occurred in each dataset?
10. Which preprocessing operations were required?
11. Which model performed best for each application?

12. Which evaluation metrics were most appropriate?
13. Did the best model have the best deployment characteristics?
14. Which application was easiest to deploy?
15. Which application was most difficult to deploy?
16. What forms of data leakage were considered?
17. What limitations remain in each system?
18. What would you improve with additional time?

17.1 Cross-Application Comparison

Students should complete the following table.

Aspect	**Diabetes**	**House Price**	**Customer Behavior**
Problem type			
Observation			
Target			
Input representation			
Data-quality issues			
Best model			
Main metric			
Web deployment			
Mobile deployment			
Main limitation			

18 Conclusion

Summarize what was learned from developing the three intelligent applications.

The conclusion should return to the central principle of the assignment:

$$\boxed{\text{Raw Data} \rightarrow \text{Clean} \rightarrow \text{Represent} \rightarrow \text{Learn} \rightarrow \text{Evaluate} \rightarrow \text{Persist} \rightarrow \text{Deploy}}$$

Students should explain that an intelligent system is not simply a trained machine-learning model.

A deployable intelligent system combines data, representation, learning, evaluation, software, deployment, and user interaction.

The conclusion should contain:

1. the main lesson learned;
2. the most important technical challenge;
3. the most important data-representation issue;
4. the most important ML lesson;
5. the most important deployment lesson;
6. one proposed improvement for future work.

A Appendix A — Repository Structure

Students should organize the project approximately as follows:

```
Assignment_02/
|
+-- diabetes/
|   +-- data/
|   +-- notebook/
|   +-- model/
|   |   +-- preprocessor.joblib
|   |   +-- model.joblib
|   +-- api/
|   +-- web/
|   +-- mobile/
|   +-- requirements.txt
|
+-- house_price/
|   +-- data/
|   +-- notebook/
|   +-- model/
|   |   +-- preprocessor.joblib
|   |   +-- model.joblib
|   +-- api/
|   +-- web/
|   +-- mobile/
|   +-- requirements.txt
|
+-- customer_behavior/
|   +-- data/
|   +-- notebook/
|   +-- model/
|   |   +-- preprocessor.joblib
|   |   +-- model.joblib
|   +-- api/
|   +-- web/
|   +-- mobile/
|   +-- requirements.txt
|
+-- report/
|   +-- Assignment_02.pdf
|
+-- README.md
```

B Appendix B — Required Notebook Structure

Every application notebook should contain the following sections:

1. Problem definition
2. Dataset source
3. Dataset loading

4. Dataset inspection
5. Data-quality analysis
6. Missing-value analysis
7. Duplicate analysis
8. Invalid-value analysis
9. Outlier analysis
10. Exploratory data analysis
11. Feature types
12. Data representation
13. Feature engineering
14. Train/test split
15. Preprocessing pipeline
16. Baseline model
17. Model training
18. Model comparison
19. Evaluation
20. Error analysis
21. Model selection
22. Model persistence
23. Inference test

C Appendix C — Required API Structure

Each application should expose a prediction endpoint similar to:

`/predict`

The general API workflow is:

$$\text{JSON Input} \rightarrow \text{Validation} \rightarrow \text{Preprocessing} \rightarrow \text{Model} \rightarrow \text{JSON Output}.$$

Example conceptual response:

```
{
    "prediction": "...",
    "probability": 0.91
}
```

The exact API implementation may differ depending on the application and framework.

D Appendix D — Required Web Report Template

For each application, report:

1. Web framework.
2. API endpoint.
3. Input variables.
4. Validation rules.
5. Preprocessing used.
6. Loaded model.
7. Example request.
8. Example response.
9. Screenshot of input interface.
10. Screenshot of prediction result.

Web Application — [Application Name]
Framework: _____
Endpoint: _____
Input: _____
Output: _____

Insert web application screenshots here.

E Appendix E — Required Mobile Report Template

For each application, report:

1. Mobile framework.
2. Platform.
3. Input screen.
4. API endpoint.
5. Request format.
6. Response format.
7. Prediction display.
8. Screenshots.

Mobile Application — [Application Name]
Framework: _____
Platform: _____
API: _____

Insert mobile screenshots here.

F Appendix F — Final Submission Checklist

Requirement	Done
Three Kaggle datasets selected	☐
Diabetes application completed	☐
House-price application completed	☐
Customer-behavior application completed	☐
Problem definition completed for all three	☐
Dataset structure analyzed	☐
Data quality analyzed	☐
Missing values investigated	☐
Duplicates investigated	☐
Invalid values investigated	☐
Outliers investigated	☐
Data representation explained	☐
Numerical features identified	☐
Categorical features identified	☐
Feature engineering explained	☐
EDA completed	☐
Train/test split performed correctly	☐
Data leakage considered	☐
Preprocessing pipeline implemented	☐
At least four ML models compared per application	☐
Appropriate metrics reported	☐
Confusion matrix reported where appropriate	☐
Best model selected and justified	☐
Error analysis completed	☐
Model saved	☐
Preprocessing saved	☐
Inference tested	☐
REST API implemented	☐
Web application implemented	☐
Mobile application implemented	☐
Diabetes web screenshots included	☐
Diabetes mobile screenshots included	☐
House-price web screenshots included	☐
House-price mobile screenshots included	☐
Customer-behavior web screenshots included	☐
Customer-behavior mobile screenshots included	☐
Reproducibility information included	☐
Source code submitted	☐
README included	☐
Final comparative discussion included	☐
Final PDF submitted	☐